



Despliegue Básico de *Pipelines* en Kedro

Exportación y Empaquetado de Proyectos Kedro

El **empaquetado en Kedro** es el proceso de crear una distribución autónoma de tu proyecto que puede ser instalada y ejecutada en diferentes entornos sin necesidad de acceso al código fuente original.

Este proceso genera dos componentes principales:

1. **Archivo Wheel (.whl):** Contiene los archivos binarios de Python del proyecto empaquetado.
2. **Archivo de Configuración (.tar.gz):** Contiene todas las configuraciones del proyecto.

Un archivo **wheel** es un archivo en formato ZIP con un nombre especial y la extensión **.whl**. Contiene una distribución simple de Python y los archivos del proyecto (Python Packaging Authority, s.f.).

El comando principal para empaquetar un proyecto Kedro es: **kedro package** ejecutado desde la carpeta raíz del proyecto (recuerda activar tu ambiente virtual y cambiar a la carpeta del proyecto de Kedro).

Para el proyecto creado en la lección 4, llamado **kedrotlg** el proceso sería:

```
PROBLEMS 1 OUTPUT TERMINAL QUERY RESULTS PORTS DEBUG CONSOLE
• PS D:\workspace\itsem\tlg\kedro> .venv\Scripts\activate
• (.venv) PS D:\workspace\itsem\tlg\kedro> cd kedrotlg
• (.venv) PS D:\workspace\itsem\tlg\kedro\kedrotlg> kedro package
[06/08/25 18:56:40] INFO Using 'conf\logging.yml' as logging configuration. You can change this by setting the KEDRO_LOGGING_CONFIG environment variable accordingly.
'D:\workspace\itsem\tlg\kedro\.venv\Scripts\python.exe' -m build --wheel --outdir dist
* Creating isolated environment: venv+pip...
* Installing packages in isolated environment:
  - setuptools
* Getting build dependencies for wheel...
running egg_info
creating src\kedrotlg.egg-info
writing src\kedrotlg.egg-info\PKG-INFO
writing dependency_links to src\kedrotlg.egg-info\dependency_links.txt
writing entry points to src\kedrotlg.egg-info\entry_points.txt
writing requirements to src\kedrotlg.egg-info\requires.txt
writing top-level names to src\kedrotlg.egg-info\top_level.txt
writing manifest file 'src\kedrotlg.egg-info\SOURCES.txt'
reading manifest file 'src\kedrotlg.egg-info\SOURCES.txt'
writing manifest file 'src\kedrotlg.egg-info\SOURCES.txt'
* Building wheel...
running bdist_wheel
running build
running build_py
creating build\lib\kedrotlg
copying src\kedrotlg\pipeline_registry.py -> build\lib\kedrotlg
copying src\kedrotlg\settings.py -> build\lib\kedrotlg
copying src\kedrotlg\__init__.py -> build\lib\kedrotlg
copying src\kedrotlg\__main__.py -> build\lib\kedrotlg
creating build\lib\kedrotlg\pipelines
copying src\kedrotlg\pipelines\__init__.py -> build\lib\kedrotlg\pipelines
creating build\lib\kedrotlg\pipelines\preprocess_data
copying src\kedrotlg\pipelines\preprocess_data\nodes.py -> build\lib\kedrotlg\pipelines\preprocess_data
copying src\kedrotlg\pipelines\preprocess_data\pipeline.py -> build\lib\kedrotlg\pipelines\preprocess_data
copying src\kedrotlg\pipelines\preprocess_data\__init__.py -> build\lib\kedrotlg\pipelines\preprocess_data
running egg_info
writing src\kedrotlg.egg-info\PKG-INFO
```

Figura 1. Empaquetado de un proyecto Kedro.

Esto creará 2 archivos dentro de la subcarpeta **dist**.

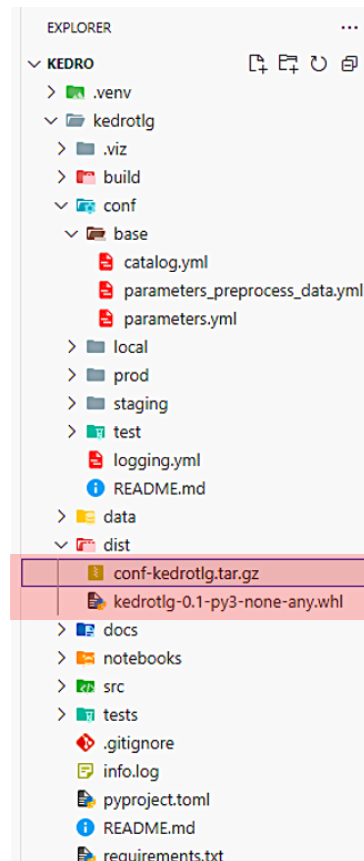


Figura 2. Archivos en la carpeta **dist**.

El archivo **.whl** contiene:

- Todo el código fuente de **src/**
- Dependencias especificadas en **requirements.txt**
- Metadatos del proyecto de **pyproject.toml**
- *Scripts* ejecutables definidos

El archivo **.tar.gz** incluye:

- Configuraciones de **conf/base/**
- Parámetros, catálogos y configuraciones de *logging*.

En este proceso se excluyen:

- Configuraciones locales (`conf/local/`)
- Carpetas de datos (`data/`)
- Archivo de configuración de proyecto `pyproject.toml`

Este empaquetamiento permite que se pueda distribuir el proyecto para ejecutarse en cualquier otro lado, por ejemplo, un servidor remoto. Cuando se distribuya, el proyecto debe ser ejecutado desde dentro de un directorio que contenga el archivo `pyproject.toml`, la subcarpeta `conf/`, y si el proyecto lo requiere, la subcarpeta `data/`. Esto quiere decir que estos directorios se tendrán que crear de manera manual.

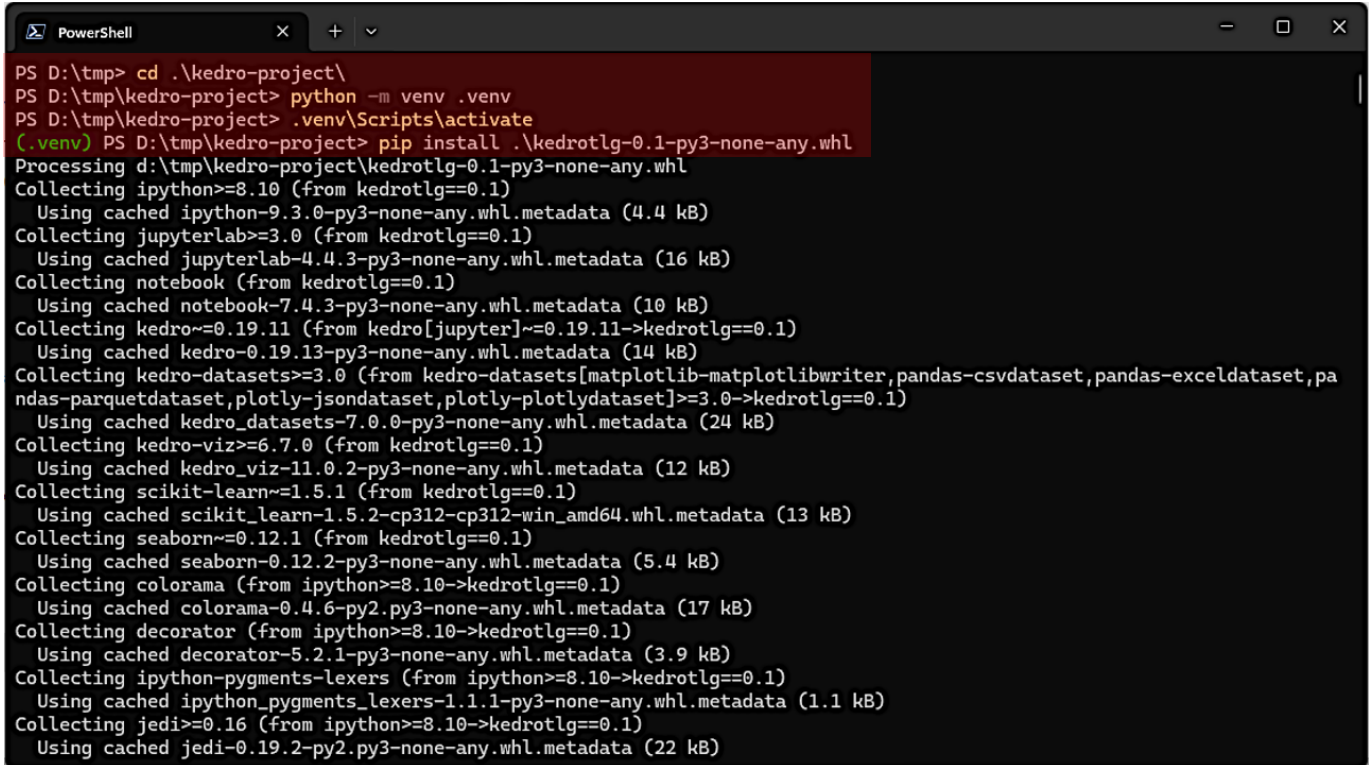
Para instalar el archivo `.whl` en el servidor remoto, se requiere tener Python y pip, aunque no es necesario tener Kedro instalado.

```
pip install <nombre del archivo wheel>
```

A menos que creamos un ambiente virtual de Python, esto instalará el proyecto de Kedro a nivel sistema.

Vamos a copiar los archivos generados del proyecto `kedrotlg`, a otra carpeta. En el ejemplo es una carpeta llamada `D: \tmp\kedro-project` (en Windows), pero puede ser cualquier carpeta dentro de tu sistema operativo.

En la siguiente terminal podemos ver cómo se crearía un ambiente virtual, como se activaría y como se instalaría el archivo **.whl**, todo dentro de la carpeta mencionada anteriormente.



```
PS D:\tmp> cd .\kedro-project\  
PS D:\tmp\kedro-project> python -m venv .venv  
PS D:\tmp\kedro-project> .venv\Scripts\activate  
(.venv) PS D:\tmp\kedro-project> pip install .\kedrotlg-0.1-py3-none-any.whl  
Processing d:\tmp\kedro-project\kedrotlg-0.1-py3-none-any.whl  
Collecting ipython>=8.10 (from kedrotlg==0.1)  
  Using cached ipython-9.3.0-py3-none-any.whl.metadata (4.4 kB)  
Collecting jupyterlab>=3.0 (from kedrotlg==0.1)  
  Using cached jupyterlab-4.4.3-py3-none-any.whl.metadata (16 kB)  
Collecting notebook (from kedrotlg==0.1)  
  Using cached notebook-7.4.3-py3-none-any.whl.metadata (10 kB)  
Collecting kedro~0.19.11 (from kedro[jupyter]~0.19.11->kedrotlg==0.1)  
  Using cached kedro-0.19.13-py3-none-any.whl.metadata (14 kB)  
Collecting kedro-datasets>=3.0 (from kedro-datasets[matplotlib-matplotlibwriter,pandas-csvdataset,pandas-exceldataset,pandas-parquetdataset,plotly-jsondataset,plotly-plotlydataset]>=3.0->kedrotlg==0.1)  
  Using cached kedro_datasets-7.0.0-py3-none-any.whl.metadata (24 kB)  
Collecting kedro-viz>=6.7.0 (from kedrotlg==0.1)  
  Using cached kedro_viz-11.0.2-py3-none-any.whl.metadata (12 kB)  
Collecting scikit-learn~1.5.1 (from kedrotlg==0.1)  
  Using cached scikit_learn-1.5.2-cp312-cp312-win_amd64.whl.metadata (13 kB)  
Collecting seaborn~0.12.1 (from kedrotlg==0.1)  
  Using cached seaborn-0.12.2-py3-none-any.whl.metadata (5.4 kB)  
Collecting colorama (from ipython>=8.10->kedrotlg==0.1)  
  Using cached colorama-0.4.6-py2.py3-none-any.whl.metadata (17 kB)  
Collecting decorator (from ipython>=8.10->kedrotlg==0.1)  
  Using cached decorator-5.2.1-py3-none-any.whl.metadata (3.9 kB)  
Collecting ipython-pygments-lexers (from ipython>=8.10->kedrotlg==0.1)  
  Using cached ipython_pygments_lexers-1.1.1-py3-none-any.whl.metadata (1.1 kB)  
Collecting jedi>=0.16 (from ipython>=8.10->kedrotlg==0.1)  
  Using cached jedi-0.19.2-py2.py3-none-any.whl.metadata (22 kB)
```

Figura 3. Creación de un ambiente virtual.

Una vez que el proceso de instalación termine, descomprimos el archivo **conf-kedrotlg.tar.gz** dentro de la misma carpeta. Esto generará una carpeta llamada **conf/**, que contiene la configuración requerida para ejecutar el proyecto. Aquí deberemos modificar los archivos de configuración para reflejar el ambiente de producción. Si la configuración del proyecto en **catalog.yml** lo requiere, habrá que crear la carpeta **data/**, **conf/local/** y sus subcarpetas.

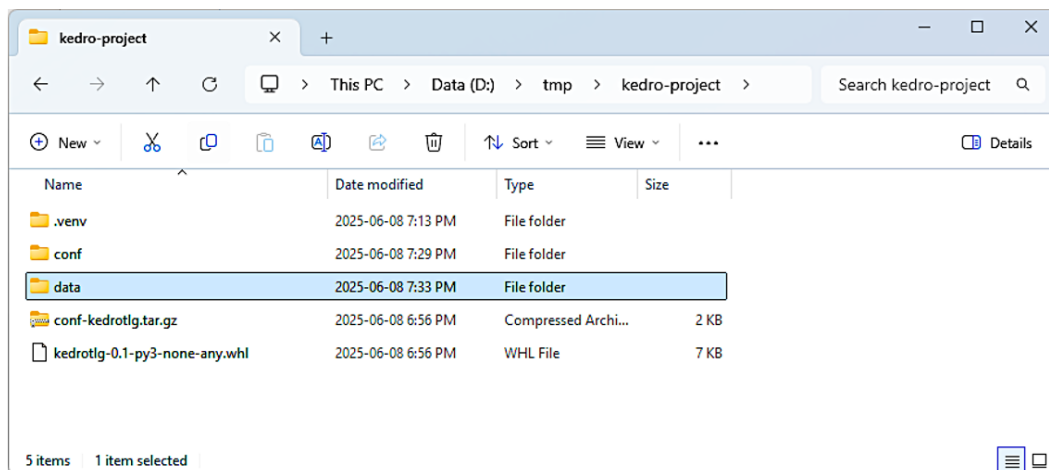


Figura 4. Carpetas de configuración.

Los archivos **.whl** y **.tar.gz** se pueden eliminar posteriormente. El proyecto se podrá ejecutar directamente como un módulo de Python.

```
python -m kedrotlg
```

Dentro de la carpeta donde están **conf/** y **data/**. Si se creó un ambiente virtual, recuerda activarlo.

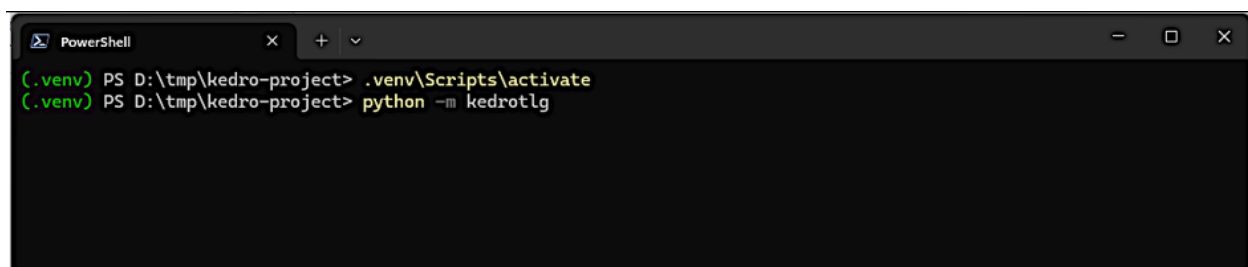


Figura 5. Activación del ambiente virtual.

En algunas ocasiones, se desea personalizar el empaquetado. Esto se hace directamente en el archivo `pyproject.toml` en la carpeta original con el código fuente del proyecto.

Por ejemplo, para especificar de manera explícita la **versión** del paquete, se puede agregar el parámetro `version` (recuerda borrar `dynamic = ["versión"]`)

```
kedrotlg > pyproject.toml > {} project > [ ] dynamic
1  [build-system]
2  requires = ["setuptools"]
3  build-backend = "setuptools.build_meta"
4
5  [project]
6  requires-python = ">=3.9"
7  name = "kedrotlg"
8  version = "0.2"
9  readme = "README.md"
10 dynamic = ["version"]
11 dependencies = [
12     "ipython>=8.10",
13     "jupyterlab>=3.0",
14     "notebook",
15     "kedro[jupyter]~=0.19.11",
16     "kedro-datasets[pandas-csvdataset, pandas-exceldataset, pandas-pa",
17     "kedro-viz>=6.7.0",
18     "scikit-learn~=1.5.1",
19     "seaborn~=0.12.1"
20 ]
21
22 [project.scripts]
23 "kedrotlg" = "kedrotlg. main :main"
```

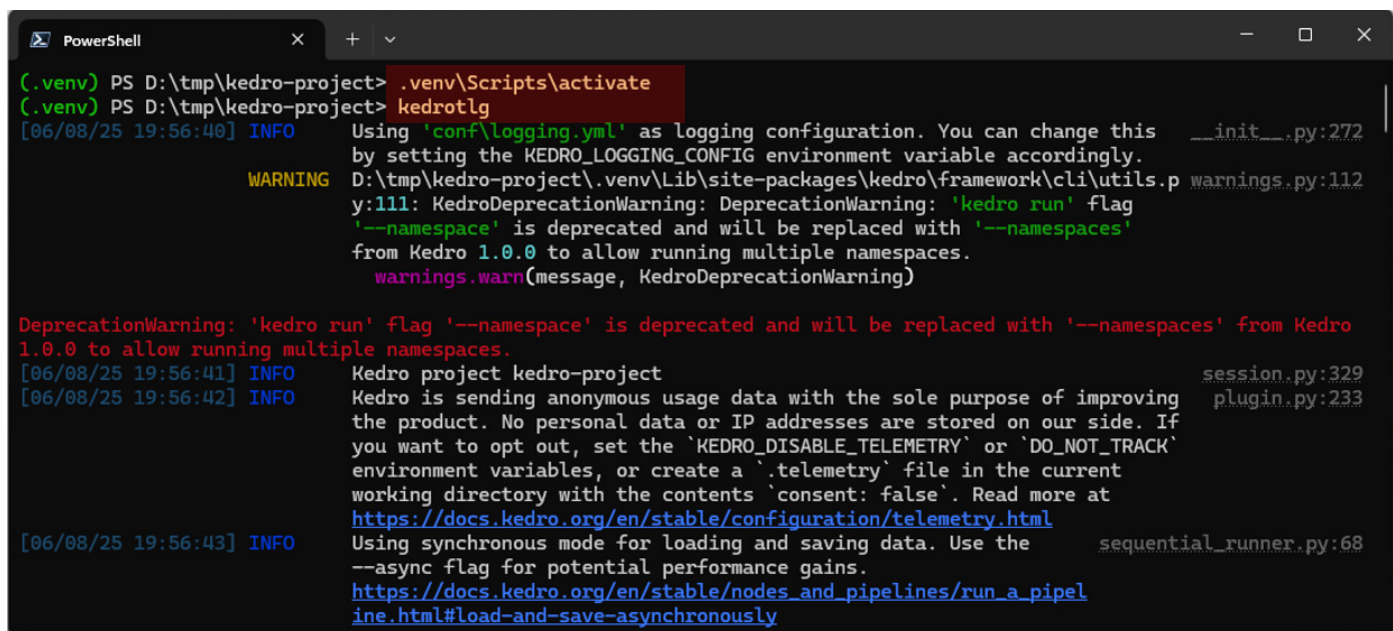
Figura 6. Parámetro `version`.

Generación de *scripts* ejecutables

Los ***scripts* ejecutables** permiten que un proyecto Kedro se comporte como una aplicación de línea de comandos, facilitando la integración con sistemas de orquestación y automatización.

Las versiones más nuevas de Kedro generan esta aplicación por defecto, permitiendo ejecutar el proyecto directamente.

Por ejemplo, para el paquete instalado anteriormente, podemos ejecutar el proyecto de la siguiente manera:



```
PowerShell
(.venv) PS D:\tmp\kedro-project> .venv\Scripts\activate
(.venv) PS D:\tmp\kedro-project> kedrotlg
[06/08/25 19:56:40] INFO Using 'conf\logging.yml' as logging configuration. You can change this by setting the KEDRO_LOGGING_CONFIG environment variable accordingly.
WARNING D:\tmp\kedro-project\.venv\Lib\site-packages\kedro\framework\cli\utils.py:111: KedroDeprecationWarning: DeprecationWarning: 'kedro run' flag '--namespace' is deprecated and will be replaced with '--namespaces' from Kedro 1.0.0 to allow running multiple namespaces.
DeprecationWarning: 'kedro run' flag '--namespace' is deprecated and will be replaced with '--namespaces' from Kedro 1.0.0 to allow running multiple namespaces.
[06/08/25 19:56:41] INFO Kedro project kedro-project
[06/08/25 19:56:42] INFO Kedro is sending anonymous usage data with the sole purpose of improving the product. No personal data or IP addresses are stored on our side. If you want to opt out, set the 'KEDRO_DISABLE_TELEMETRY' or 'DO_NOT_TRACK' environment variables, or create a '.telemetry' file in the current working directory with the contents 'consent: false'. Read more at https://docs.kedro.org/en/stable/configuration/telemetry.html
[06/08/25 19:56:43] INFO Using synchronous mode for loading and saving data. Use the --async flag for potential performance gains.
https://docs.kedro.org/en/stable/nodes_and_pipelines/run_a_pipeline.html#load-and-save-asynchronously
```

Figura 7. Generación de *scripts*.

Visión general de ejecución en entornos de producción

Consideraciones para producción

La ejecución en producción requiere considerar múltiples aspectos que tal vez no son prioritarios durante la etapa de desarrollo:

1. Confiabilidad y Recuperación:

- Manejo de errores robusto, esto quiere decir uso de excepciones donde sea necesario dentro de los nodos y funciones de apoyo.
- Reintentos automáticos en caso de errores o problemas.
- Puntos de recuperación (*checkpoints*).
- Monitoreo continuo.

2. Escalabilidad:

- Capacidad de manejar volúmenes de información crecientes.
- Paralelización de tareas.
- Distribución de carga.
- Optimización de recursos.

3. Observabilidad:

- *Logging* detallado.
- Métricas de rendimiento.
- Alertas automáticas.
- Trazabilidad de datos.

4. Seguridad:

- Gestión de credenciales.
- Acceso controlado.
- Auditoría de cambios.
- Cumplimiento normativo.

Ahora bien, con el auge de las plataformas basadas en nube, es posible que el despliegue de estos proyectos se realice en algún servicio de este tipo. Para esto debemos considerar:

Local vs Cloud

En una ejecución local existen:

Ventajas:

- **Control total:** Acceso completo al hardware y software.
- **Latencia baja:** Debido a que no hay transferencia de datos a servicios externos.
- **Costo predecible:** Sin costos variables por uso.
- **Privacidad:** Datos nunca salen del entorno controlado.

Desventajas:

- **Escalabilidad limitada:** Restringida por hardware local.
- **Mantenimiento:** Responsabilidad total de infraestructura.
- **Disponibilidad:** Sin redundancia automática.
- **Respaldos:** Gestión manual de respaldos.

También existen en la ejecución en la nube:

Ventajas:

- **Escalabilidad automática:** Recursos bajo demanda.
- **Alta disponibilidad:** Disponibilidad de redundancia automática.
- **Servicios gestionados:** Disponibilidad de servicios gestionados por el proveedor, como bases de datos, colas, almacenamiento, respaldos, etcétera.
- **Alcance global:** Despliegue en múltiples regiones para los proveedores más grandes.

Desventajas:

- **Costos variables:** Dependientes del uso, el costeo puede ser complejo.
- **Latencia de red:** Transferencia de datos, esto afecta tanto el desempeño como el costeo.
- **Vendor lock-in:** Dependencia del proveedor que puede dificultar la migración a otro proveedor.
- **Complejidad:** Configuración más compleja y que puede requerir personal especializado.

I Docker, Airflow, Prefect, etcétera

Un enfoque común para el despliegue, especialmente en entornos distribuidos o en la nube, es “**contenerizar**” el proyecto Kedro, a menudo utilizando **Docker**. Un **contenedor** es una unidad de software que empaqueta una aplicación y todas sus dependencias (bibliotecas, configuraciones, entre otros) para que pueda ejecutarse de forma rápida y confiable en cualquier entorno informático compatible.

Un contenedor se puede pensar como una caja estandarizada que contiene todo lo que una aplicación necesita para funcionar, independientemente de dónde se abra esa caja. Esto elimina problemas de compatibilidad y facilita el despliegue de software.

Por otro lado, también es posible desplegar *pipelines* de Kedro en **plataformas de orquestación**, que son herramientas de software diseñadas para automatizar, coordinar y gestionar flujos de trabajo complejos que involucran múltiples tareas, sistemas y aplicaciones.

Kedro se integra con diversas herramientas de orquestación, algunas de las plataformas de orquestación más comunes son:

- **Apache Airflow**, es una popular plataforma *open source* de gestión de flujos. Al igual que Kedro, Airflow usa el concepto de grafo acíclico dirigido, (DAG). La estrategia general es ejecutar cada nodo de Kedro como una tarea de Airflow, generando el DAG correspondiente en Airflow. Esta plataforma está soportada por prácticamente todos los proveedores de nube más grandes (Kedro, s.f.).
- **Prefect**, es una herramienta *open source* de orquestación de flujos de trabajo (*workflows*) escrita en Python. Permite al desarrollador construir, programar y monitorear *pipelines* de datos robustos y resilientes, especialmente en el contexto de la ingeniería de datos y el *Machine Learning* (MLOps). (Kedro, s.f.)

Adicionalmente, la comunidad ha creado *plugins* para integrar Kedro con plataformas propietarias, como Azure ML Studio, Google Vertex AI, AWS Sagemaker, Databricks, entre otros.

| Antes de terminar...

El despliegue y la puesta en producción de proyectos Kedro representa una fase crítica que determina el éxito real de nuestros *pipelines* de datos.

A lo largo de esta lección, hemos explorado las herramientas y estrategias esenciales para llevar nuestros proyectos del entorno de desarrollo a la producción de manera efectiva.

Sin embargo, el despliegue es una tarea compleja que generalmente requiere el involucramiento de los equipos de infraestructura y ciberseguridad.

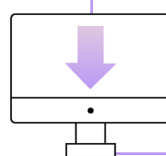
Intenta descubrir en tu organización como sería la puesta en producción de un *pipeline* de Kedro:

- ¿En qué plataforma sería?, ¿en un servidor remoto?, ¿en la nube?
- ¿Qué equipos técnicos deberán involucrarse?

Bibliografía

Kedro. (s.f.) *A toolbox for production-ready data science* [Una caja de herramientas para ciencia de datos lista para producción]. Kedro. Recuperado el 11 de septiembre de 2025, de: <https://docs.kedro.org/en/1.0.0/index.html>

Nota: Los enlaces a las páginas web incluidos en esta bibliografía fueron consultados y verificados por última vez el 11 de septiembre de 2025 y su disponibilidad depende del autor.



Descarga este documento en la sección “**Archivos adjuntos**” de la plataforma.